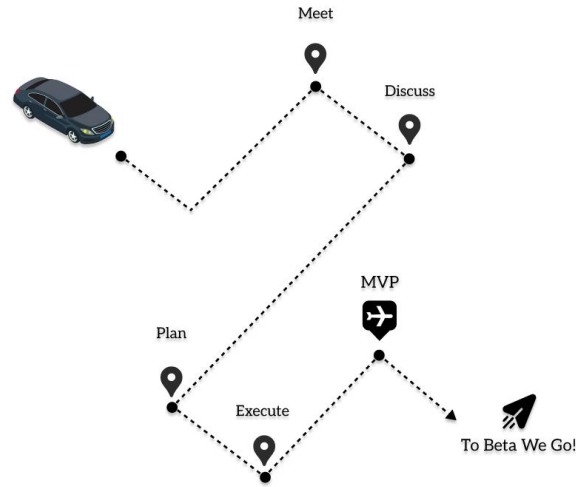




ROAD TO MVP AND BEYOND

UOB SUTAINABLE TRANSPORT

01

ABOUT US



IOAN MOIR
PROJECT MANAGER

YIDI AI



ERIK MALTBY
CLIENT LIAISON



OTGON-ERDENE



OUR PARTNER



PROLINE TAXI COMPANY

A company that offers professional, reliable, and affordable luxury taxi & chauffeur services.

COLLABORATION

Proline has been offering taxi services to the University of Bristol staff since 2019.



PROBLEM

The current booking process with the University of Bristol is inefficient, time-consuming, and heavily manual.

WHAT ARE WE TRYING TO ACHIEVE ?

STREAMLINING

Replace the manual processes through emails and phone calls with a digital experience that is **FASTER**.

TRANSPARENCY

Provide passengers with live information about their booking status and progress **FASTER**.

AUTOMATION

Sending booking details to the department's finance team for **FASTER** approval.

NOTICE A THEME ?



SAVE



TIME !



USER



STORIES



PASSENGER

I want the booking approval process to be faster and have transparent communication with the assigned driver to **save time**.

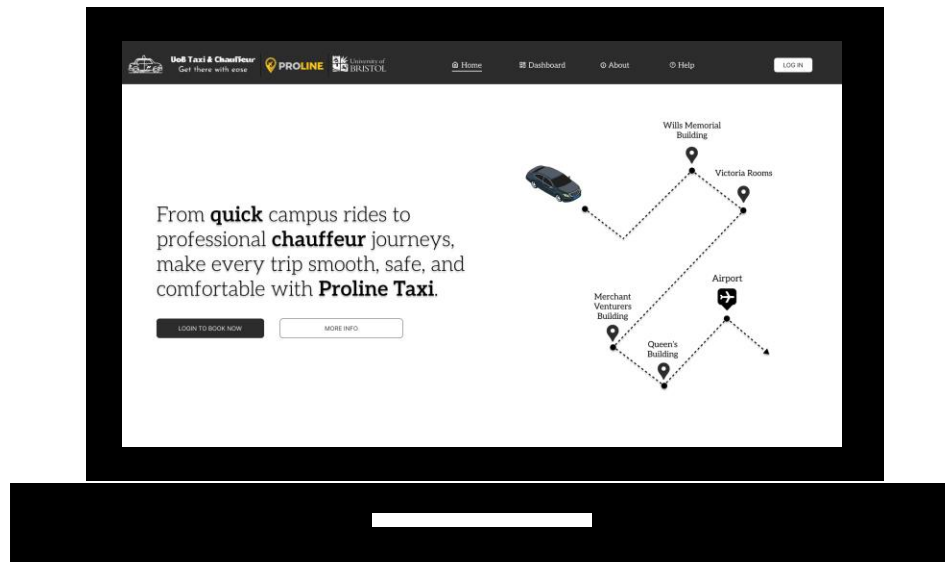
HEAD OF DEPARTMENT

I want to view the key details of a booking instantly and approve or reject it **quickly**.

PROLINE

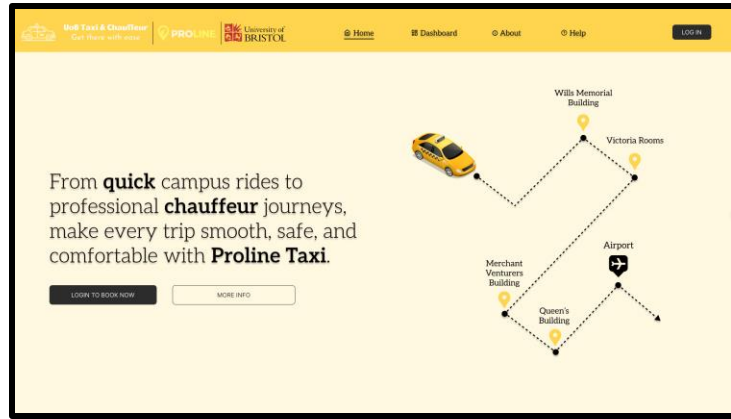
We want to automate the current laborious back-and-forth booking process to **save time**.

DEMO

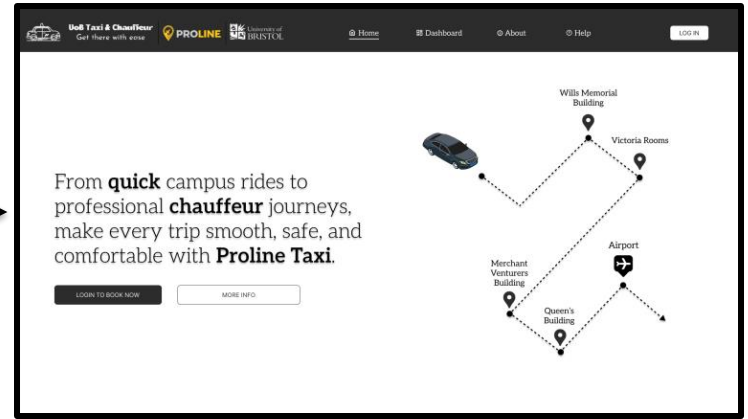


02 DESIGN EVOLUTION

TOO TAXI-EY ?



- Monochrome
- More luxurious feeling
- Less distractions



BOOKING PAGE

 **UoB Taxi & Chauffeur**
Get there with ease

 **PROLINE**

 **University of BRISTOL**

[Home](#) [Dashboard](#) [About](#) [Help](#)



BOOKING DETAILS

Common pick-up locations

Choose

Manually Enter ☐

Royal Fort House

Drop-off location

Stuart House

Name

Surname

Phone number

+44

Email

Additional information

CONFIRM BOOKING





 **UoB Taxi & Chauffeur**
Get there with ease

 **University of BRISTOL**

 **PROLINE**

[Home](#) [Dashboard](#) [About](#) [Help](#)



BOOKING DETAILS

Common pick-up locations

Manually Enter ☐ Flight ☐ Via ☐

Drop-off location

Temple Quarter Enterprise Campus, Bristol

☐ Return trip

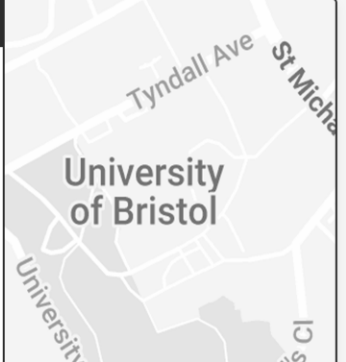
Pick-up date and time

dd/mm/yyyy

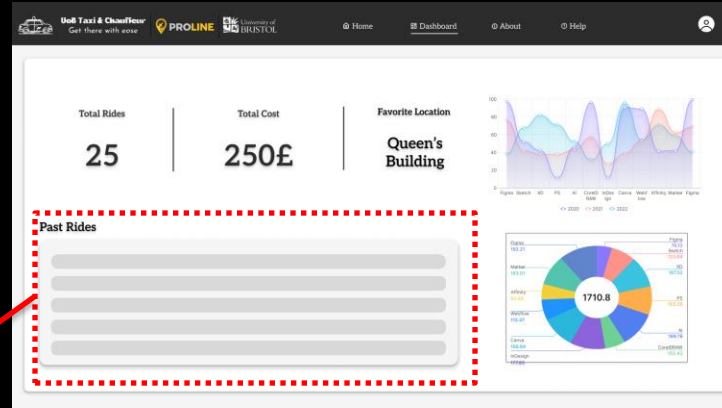
--:--

Name

Surname



DASHBOARD PAGE (HOD & USER)



USER HOME-PAGE

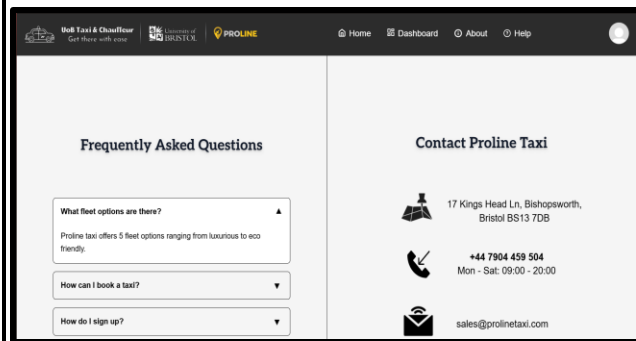
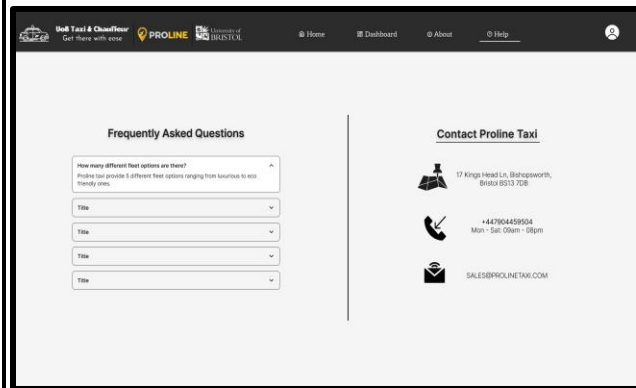
The User Home-Page displays a table of bookings under the heading "MY BOOKINGS". A red arrow points from the "Past Rides" section of the HOD dashboard to the "Home" link in the navigation bar.

Time Created	From	To	Booking Status	Operation
11/15/2025, 5:36:24 PM	Merchant Venturers Building	Will's Memorial	Cancelled	<button>View</button> <button>Cancel</button>
11/12/2025, 10:09:54 AM	Arts Complex	Physics Building	Approved	<button>View</button> <button>Cancel</button>
11/11/2025, 11:31:11 PM	Bristol Airport	Will's Memorial	Cancelled	<button>View</button> <button>Cancel</button>
11/11/2025, 10:50:04 PM	Queens Building	Heathrow Airport	Cancelled	<button>View</button> <button>Cancel</button>

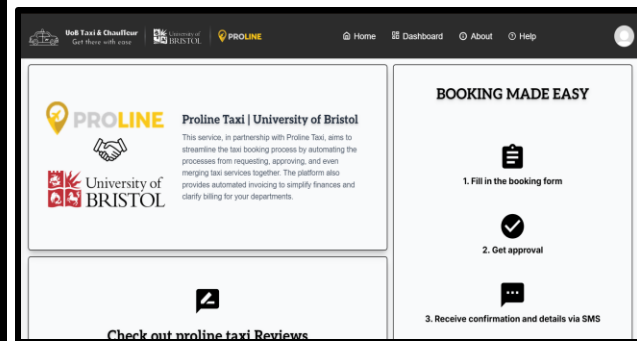
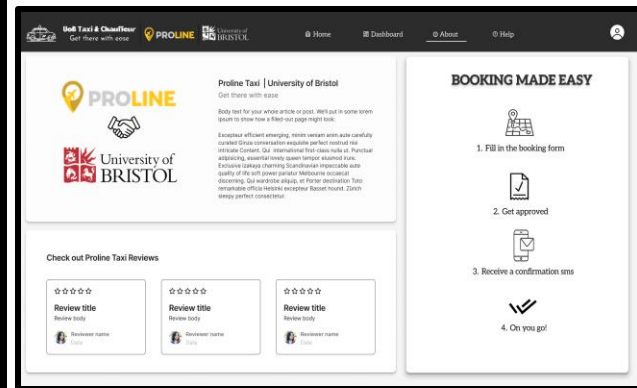
HOD APPROVAL DASHBOARD

The HOD Approval Dashboard displays a table of department bookings under the heading "Department Bookings".

Time Created	From	To	Operation
11/11/2025, 4:08:39 PM	Victoria Rooms	Richmond Building	<button>View</button> <button>Approve</button> <button>Reject</button>
11/11/2025, 3:05:15 PM	Merchant Venturers Building	Physics Building	<button>View</button> <button>Approve</button> <button>Reject</button>
11/11/2025, 3:02:51 PM	Merchant Venturers Building	Richmond Building	<button>View</button> <button>Approve</button> <button>Reject</button>
11/10/2025, 2:46:52 PM	Merchant Venturers Building	Will's Memorial	<button>View</button> <button>Approve</button> <button>Reject</button>



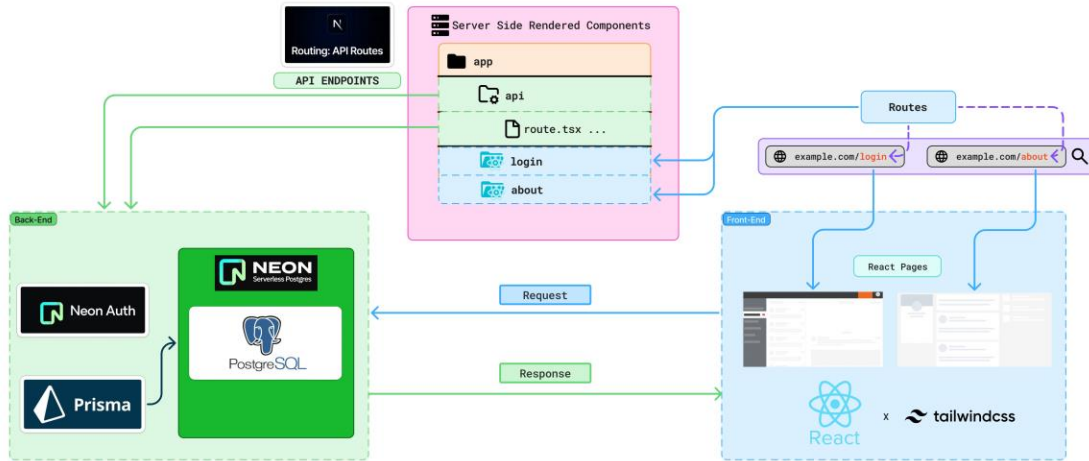
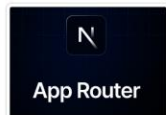
ABOUT & HELP PAGE



03

PROJECT ARCHITECTURE

PROJECT ARCHITECTURE



NEXT.JS

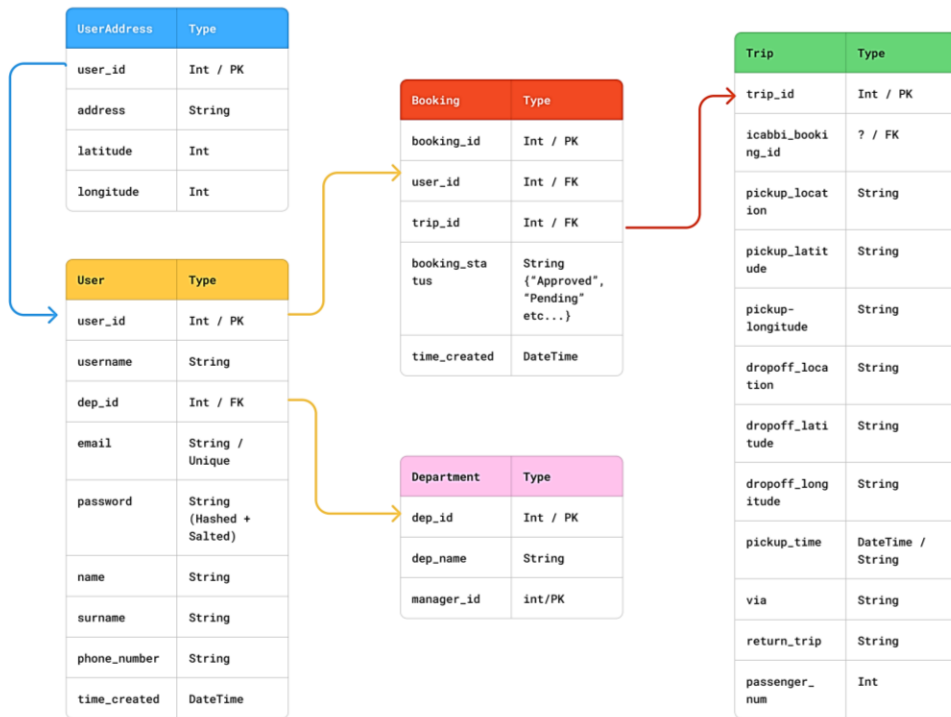
- File based routing system.
- Server Side Rendering (SSR).
- API Endpoint routes (Backend).

POSTGRESQL x NEON

- Convenient GUI (Neon Console).
- Git-like branches for DB version control.
- ORM compatibility.

PRISMA

- JS-like query syntax.
- GraphQL adjacent data fetching.
- Layer of abstraction.



ENTITY- RELATIONSHIP DIAGRAM

CI/CD AND TESTING

CONTINUOUS INTEGRATION

UNIT TESTING

Page render and API (not integration) testing Jest.

TYPE-CHECKING

Type checking with TypeScript Compiler.

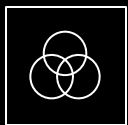
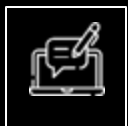
LINTING

Linting to identify small bugs and keep consistent code style.

CONTINUOUS DEPLOYMENT



USER & USABILITY TESTING PLANS



USABILITY TESTING

Allow University of Bristol staff to interact with the website and make a booking (Task-based evaluation).

RECORD THOUGHTS

Get their thoughts about the process and how it compares to the old method.

COMPARE & EVALUATE

Evaluate whether our product is more efficient by measuring the time taken and comparing with the old method.



WORKING AS A TEAM

TEAM MEETINGS

Every Wednesday, we gather and discuss how everyone's work is going

TEAM COMMUNICATION

Group chat on Teams as our central communication hub



NO SLACKING OFF

We made sure that everyone has something to work on each week

ORGANIZED KANBAN BOARD

Created clear, templated issues so everyone can choose one and know how to implement it

WORKING AS A TEAM

Hash + salt passwords in the database #170

Open

Bug

spe-uob/2025-UoBsustainableTransport

Private



ioanmoir opened yesterday

Type

Security

Description

Passwords are currently stored in plaintext. It's a good idea to store them as salted hashes using something like argonv2 instead, which makes the password unreadable to anyone that has access to the database. Each user's table entry should also have its own unique salt to prevent rainbow table attacks.

Steps to Reproduce

1. Open the Neon console.
2. Navigate to the table viewer.
3. Observe that passwords are not hashed, and that no salt field is present.

Expected Behavior

We should see random Base64 characters which would indicate the passwords are being hashed and protected correctly.

Logs or Screenshots

Conversion to JWT / volatile storage and Cookies #168

Open

Bug

spe-uob/2025-UoBsustainableTransport

Private



ioanmoir opened yesterday

Type

Security / Common practice

Description

Some parts of our authentication chain could be susceptible to attack / brute force.

Expected Behavior

Authentication tokens should:

- Be stored in a Cookie so that we don't need to send them in the body of requests (Cookies are sent with every header in GET and POST requests). Local session storage is susceptible to XSS attacks.
- Be something like a JWT token, which has more entropy / less susceptible to brute force than a UUID.
- Be stored in a volatile hashmap / array on the server, as we do not need them to persist reboot.

When a user logs in, the existing token or a new token should be created, and then the client will store this in a cookie. If the client attempts to authenticate with an invalid token, the client should destroy the cookie and prompt the user to log in again. This will remove the need to store session cookies in the database (as far as I know this is not bad practice but we don't have to do this).



UI DESIGN

The monochrome theme fitted the service, and the interface feels non-intrusive

BOOKING DETAILS

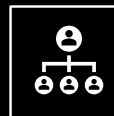
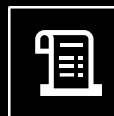
The booking form covers most essential information for a valid booking

APPROVAL SYSTEM

The approval system seems simple and intuitive



CLIENT FEEDBACK



PO & FILTERING

Feature to attach a PO number to approved bookings and filtering

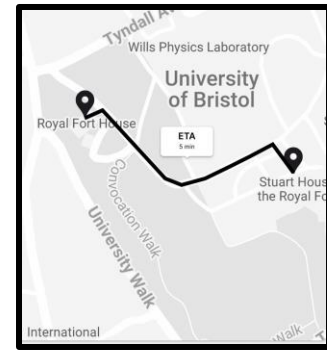
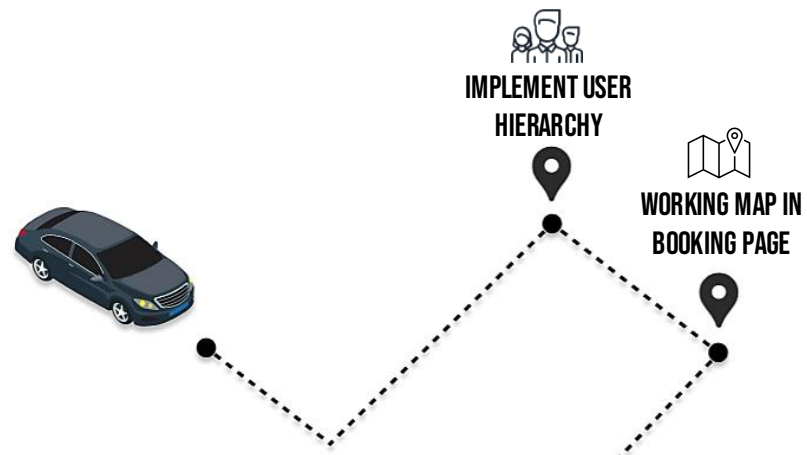
INTERACTIVE MAP

Working map to show distance, route and ETA between destinations

SUPER ADMIN

Account with super admin privilege to edit bookings at any stage of approval

04 WHAT NEXT ?




To Final We Go!

THANK YOU

ANY QUESTIONS ?